

◆ WEB3 SMART CONTRACT SECURITY REVIEW

EIP-712 SIGNATURE REPLAY

Solidity · EIP-712 · Signatures · EVM

Manual review + working proof-of-concept +
severity-rated report



● High 1

GILLES MUSY

Smart contract security researcher

Confirmed findings · Code4rena GiMu84 · Cantina GiLMu

1 Disclaimer

This document is a demonstration security review performed on SignatureClaim, a deliberately vulnerable sample authored by Gilles Musy for portfolio purposes. It is not a client engagement: the code was never deployed, holds no user funds, and the vulnerabilities it contains were intentionally introduced to illustrate review methodology, severity classification and proof-of-concept construction. The findings, proofs of concept and remediation guidance below are genuine analysis of the code as written.

A security review is a time-, resource- and expertise-bound effort to surface as many issues as possible; it cannot prove the absence of vulnerabilities. Subsequent reviews, a bug-bounty program and runtime monitoring are recommended for any production deployment.

2 Risk classification

Severity is derived from the impact of an issue and the likelihood of its occurrence.

IMPACT ▼ / LIKELIHOOD ►	HIGH	MEDIUM	LOW
HIGH	Critical	High	Medium
MEDIUM	High	Medium	Low
LOW	Medium	Low	Low

IMPACT High: significant loss of assets or harm to many users. Medium: moderate loss or harm. Low: minor loss or harm.

LIKELIHOOD High: practical under realistic on-chain conditions at low cost. Medium: conditionally incentivised but plausible. Low: needs unlikely assumptions or significant attacker stake.

3 About the target

SignatureClaim, a gasless EIP-712 claim vault.

TYPE Demonstration review on intentionally vulnerable code

METHOD Foundry proof-of-concept, vulnerable branch (master) and remediated branch (fixed)

4 Findings

HIGH H-01, Signature-malleability replay (High)

A relayar can pay any signed claim out twice. In the proof-of-concept the authorizer signs a single claim of 1,000 ether for bob ; the relayar submits the signature and then its malleable twin, and bob receives 2,000 ether. The extra 1,000 ether is drained from the vault against one authorisation. The attack needs no special role and no interaction from the signer beyond producing one legitimate signature.

ROOT CAUSE

The contract prevents replay by remembering the signature bytes:

```
bytes32 sigKey = keccak256(abi.encodePacked(r, s, v));
require(!usedSig[sigKey], "signature already used");
...
usedSig[sigKey] = true;
```

ECDSA signatures are malleable. For any valid (r, s, v) over a digest, the pair $(r, N - s, v \wedge 1)$, where N is the `secp256k1` group order, recovers the same signer for the same digest. The two encodings hash to different `sigKey` values, so the second one is seen as a fresh signature. Because the guard keys on the encoding rather than on the message, the same authorisation is accepted twice.

ATTACK

1. The authorizer signs `Claim(to = bob, amount = 1000e18)` off-chain, producing (v, r, s) with a canonical low `s`.
2. A relay calls `claim(bob, 1000e18, v, r, s)`. `bob` receives 1,000 ether.
3. The relay computes the twin: $s2 = N - s$, $v2 = v \wedge 1$. It recovers the same authorizer over the same digest.
4. The relay calls `claim(bob, 1000e18, v2, r, s2)`. `usedSig` has no entry for the twin, so the check passes and `bob` receives another 1,000 ether.

PROOF OF CONCEPT

`test/SignatureClaim.poc.t.sol`, function `test_malleability_doubleSpend`, run on `master`:

```
authorizer signed (wei): 1000000000000000000000
bob received      (wei): 2000000000000000000000
net theft        (wei): 1000000000000000000000
```

RECOMMENDATION

Two changes, both standard practice:

1. Key the replay guard on the message digest, not on the signature bytes. Every encoding of one authorisation shares the same digest, so the second submission collides.
2. Enforce EIP-2: reject `s` in the upper half of the range and reject non-canonical `v`. This refuses the malleable twin before recovery.

```
uint256 private constant HALF_N =
    0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF5D576E7357A4501DDFE92F46681B20A0;

require(uint256(s) ≤ HALF_N, "malleable signature");
require(v == 27 || v == 28, "bad v");
...
require(!usedDigest[digest], "claim already executed");
usedDigest[digest] = true;
```

This is the behaviour of OpenZeppelin's `ECDSA` library. On the `fixed` branch the same proof-of-concept shows the malleable twin rejected by the low-s check and an exact replay rejected by digest tracking; `bob` receives exactly 1,000 ether.

SEVERITY

High. The impact is direct theft of vault funds. The actor is any relayer, a role with no privilege, and the only precondition is observing one relayable signature.

5 Informational, Gas & Non-Critical

INFORMATIONAL I-01, DOMAIN_SEPARATOR cached without chain-fork handling (Informational)

`DOMAIN_SEPARATOR` is computed once in the constructor (L23-31) and stored `immutable`. If the chain forks and `block.chainid` changes, the cached separator no longer matches the active chain, so legitimately re-signed messages fail to verify. Recompute the separator when `block.chainid` differs from the deploy-time value.

NON-CRITICAL N-01, claim emits no event (Non-Critical)

`claim` (L34) releases tokens without emitting any event, leaving relayers and off-chain indexers unable to track executed claims. Emit a `Claimed(to, amount, sigKey)` event before the transfer.

INFORMATIONAL I-02, authorizer not validated in constructor (Informational)

The constructor (L20-21) does not check `_authorizer != address(0)`. With a zero authorizer, an invalid signature recovering to `address(0)` is only stopped by the `signer != address(0)` guard; validating the authorizer at construction makes the trust assumption explicit.

GAS G-01, require strings cost more than custom errors (Gas)

The three `require` reason strings (L36, L41, L44) are more expensive to deploy and revert with than custom errors on Solidity 0.8.34. Replace them with `error` declarations and `revert`.

Scope and disclaimer

`SignatureClaim` is intentionally vulnerable code written to demonstrate audit methodology end to end. It is not production code and must never be deployed. The finding above is a real vulnerability in this demo contract, reproduced with an executable proof-of-concept, not an invented severity.